

Important Concepts in Concurrent Programming (from wikipedia)

- **Concurrency vs Parallelism**

- concurrency is not only parallelism.
- concurrency
 - logically simultaneous processing
 - interleaved execution on a single processor
- parallelism
 - physically simultaneous processing
 - requires a multiprocessors or multicore system

- **Thread Safety**

- function or object is thread-safe if it behaves correctly when accessed by multiple threads at the same time without causing data corruption, unexpected results or crashes.

- **Race Condition**

- a type of flaw in an electronic or software system where the output is dependent on the sequence or timing of other uncontrollable events.
 - non-deterministic, timing dependent
 - cause data corruption, crashes
 - difficult to detect, reproduce, eliminate
- when multiple threads read and write shared data without proper synchronization, bad things (race condition) can happen.

- **Synchronization**

- coordination of simultaneous events(threads/processes) in order to obtain correct runtime order and avoid unexpected race conditions.
- ex) lock, semaphore, atomic variables, volatile

- **volatile variable**

- any read or write to it is always done directly from/to main memory - not from a thread's local cache. (volatile variable can be often used for flags or status indicators)
- Be careful: a volatile variable does not make operations atomic. (e.g. `count++;`)
 - volatile ensures visibility, not atomicity
 - volatile prevents instruction reordering around the volatile variable.

- **Mutual Exclusion**

- ensuring that no two processes or threads are in their critical section at the same time
- usual way to avoid race condition

- **Critical Section**
 - a piece of code that accesses a shared resource (data structure or device) that must not be concurrently accessed by more than one thread of execution
- **semaphore**
 - a synchronization object that maintains a count between zero and a specified maximum value. The count is decremented each time a thread completes a wait for the semaphore object and incremented each time a thread releases the semaphore. When the count reaches zero, no more threads can successfully wait for the semaphore object state to become signaled.
- **Deadlock**
 - two or more threads stop and wait for each other
 - avoiding deadlock
 - cycle in locking graph = deadlock
 - standard solution
 - canonical order for locks
 - acquire in increasing order
 - release in decreasing order
 - ensures deadlock-freedom, but not always easy to do
- **starvation**
 - some thread gets deferred forever
- **lack of fairness**
 - each thread gets a turn to make progress
- **concurrent hash map**
 - A hash table supporting full concurrency of retrievals and adjustable expected concurrency for updates.
- **Barrier**
 - a type of synchronization method. A barrier for a group of threads or processes in the source code means any thread/process must stop at this point and cannot proceed until all other threads/processes reach this barrier
- **copy on write arrays**
 - CopyOnWriteArrayList behaves as a List implementation that allows multiple concurrent reads, and for reads to occur concurrently with a write. The way it does this is to make a brand new copy of the list every time it is altered.